

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

Plataformas de adquisición de datos.

-Firebeetle 2.

-Edge 101.

-Raspberry PI.

-Guía básica de programación en Arduino IDE.

Podemos alimentarla con el puerto USB, en el proyecto disponemos de baterías de litio que se usarán para instalaciones de testeo remotas.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

Entradas:

-Digitales: 17 Entradas GPIO. GPIO11, 1 (TX0), GPIO12, 3 (RX0), GPIO10, GPIO15, GPIO14, GPIO16, GPIO13, 17, 18, 19, 21, 22, 23, GPIO6, GPIO7

Tensión de funcionamiento: 0-3,3v.

-Analógicas: 11 Canales (ADC). GPIO0, GPIO2, GPIO4, GPIO12, GPIO13, GPIO14, GPIO15, GPIO25, GPIO26, GPIO34, GPIO35.

Para sensores analógicos usar preferentemente: GPIO34, GPIO35 porque pertenecen al ADC1 y funcionan correctamente mientras el WiFi está activo.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

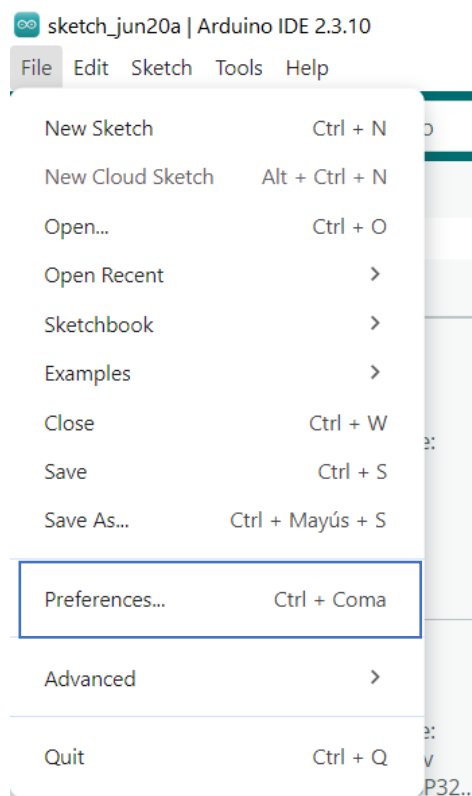
Pasos:

-Instalar el Arduino ide para programar la placa: <https://www.arduino.cc/en/Main/Software>

Downloads



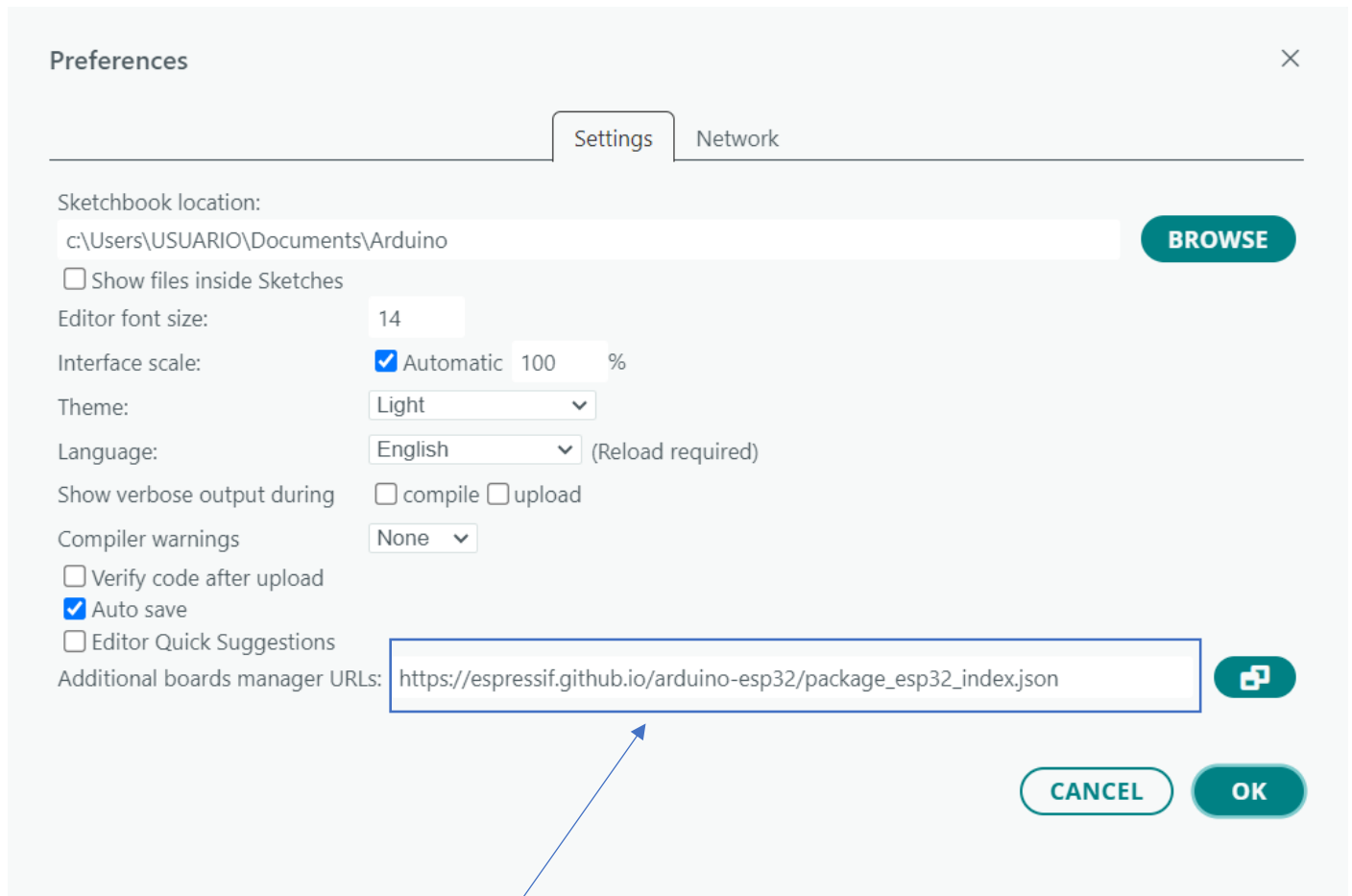
-Introducir en el Arduino ide los drivers de la placa firebeetle 2:



Nos vamos a File, Preferences.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”



Preferences

Settings Network

Sketchbook location: c:\Users\USUARIO\Documents\Arduino **BROWSE**

☐ Show files inside Sketches

Editor font size: 14

Interface scale: ☒ Automatic 100 %

Theme: Light

Language: English (Reload required)

Show verbose output during ☐ compile ☐ upload

Compiler warnings: None

☐ Verify code after upload

☒ Auto save

☐ Editor Quick Suggestions

Additional boards manager URLs: https://espressif.github.io/arduino-esp32/package_esp32_index.json **+**

CANCEL **OK**

Pulsamos donde indica y seleccionamos le archivo de nuestra placa, para ello antes debemos descargarlo de:

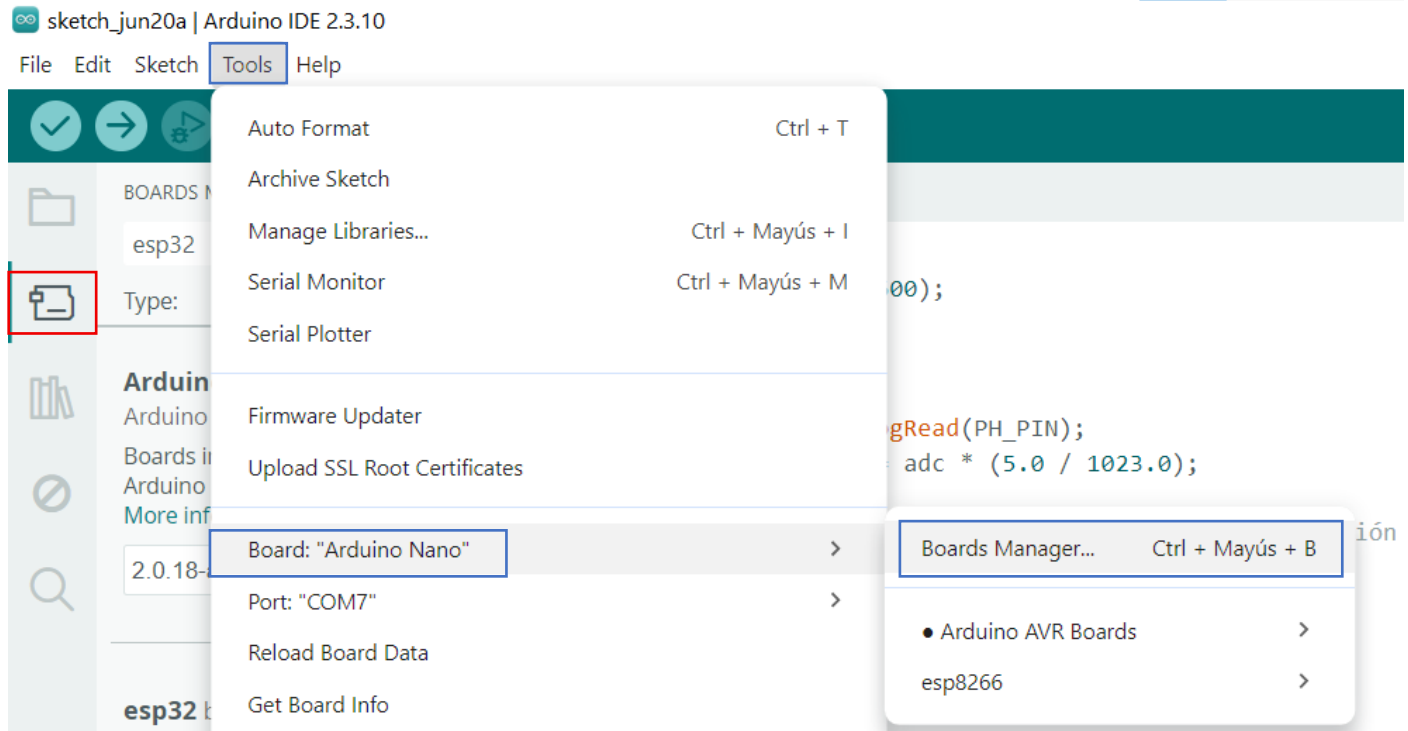
https://espressif.github.io/arduino-esp32/package_esp32_index.json

Pulsamos Ok.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

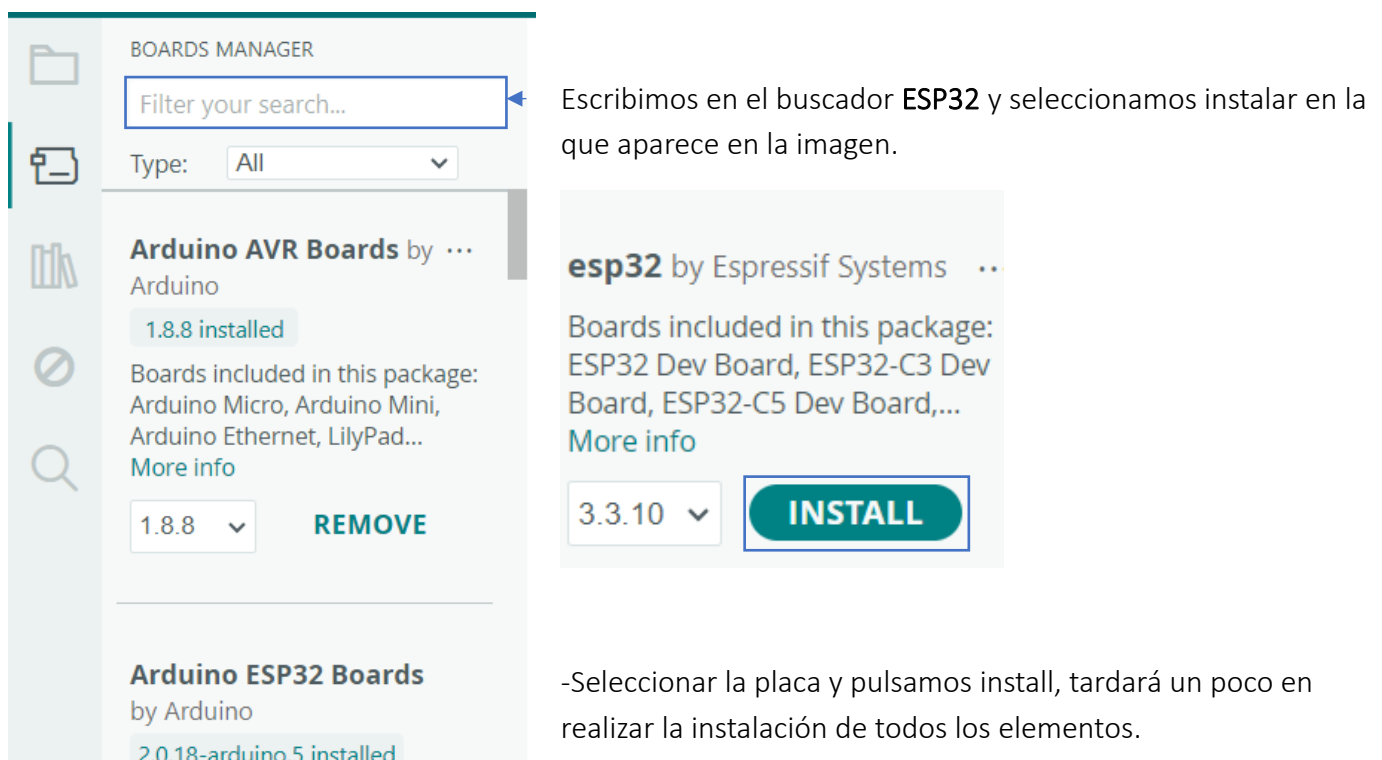
“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

-Instalar en Arduino IDE los drivers de nuestra placa:

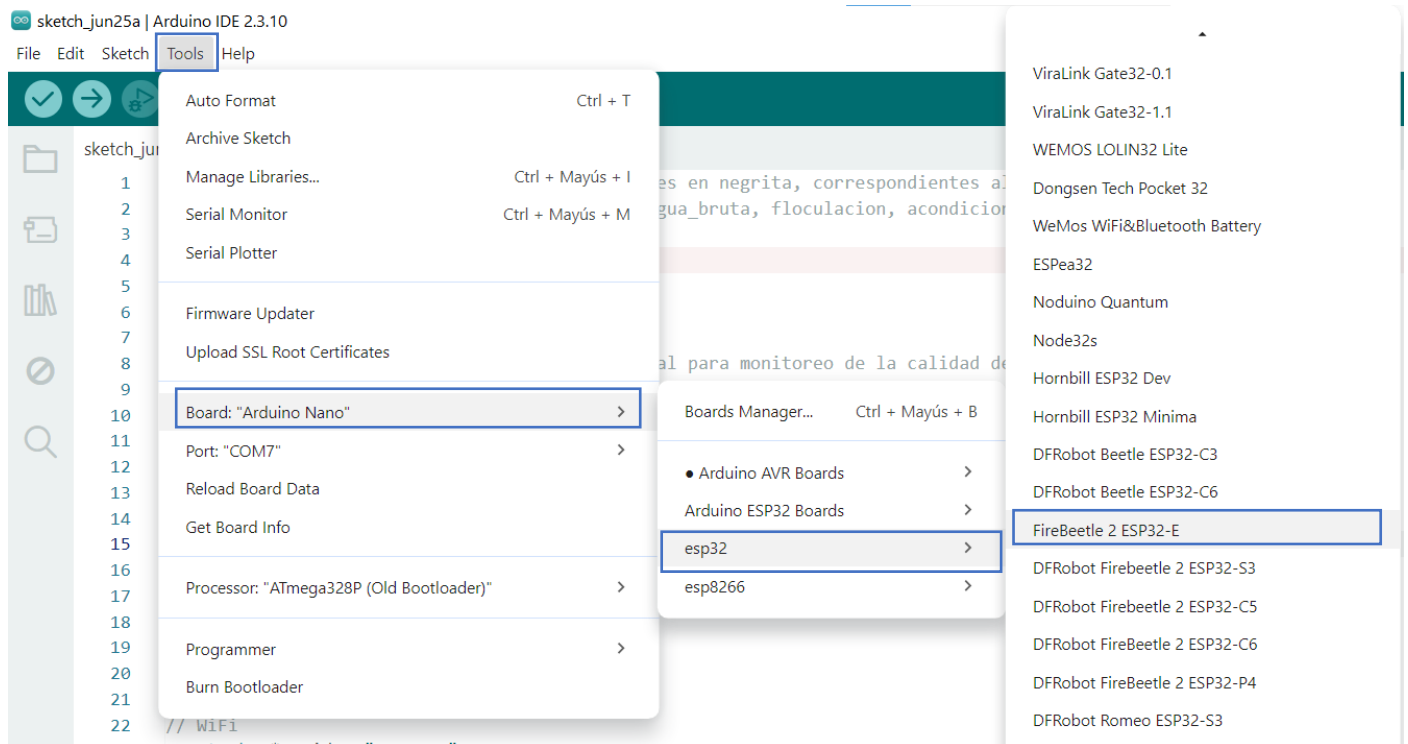


Nos vamos a Tools, Board: “Arduino XXX”, y seleccionamos Boards Manager.

O pulsamos en el icono dentro del cuadro rojo.



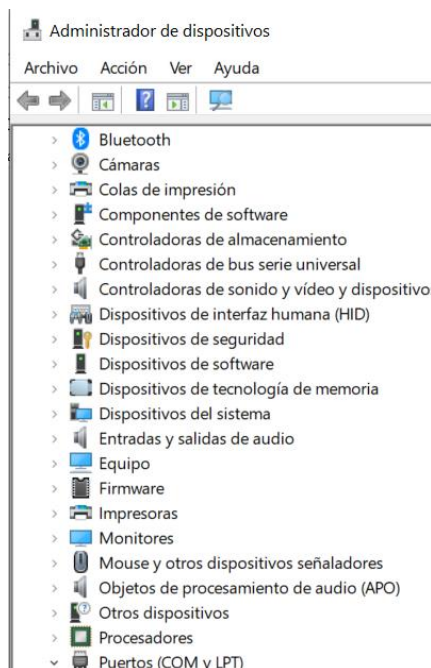
Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua
“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”



Nos vamos a Tools, Board, esp32 y seleccionamos FireBeetle 2 ESP32-E.

-Conectar la placa firebeetle 2 al ordenador.

-Debemos saber en que puerto COMM está conectada la firebeetle para ello nos vamos al administrador de dispositivos, buscamos Puertos (COM y LPT) y identificamos el que se corresponde con nuestra conexión.

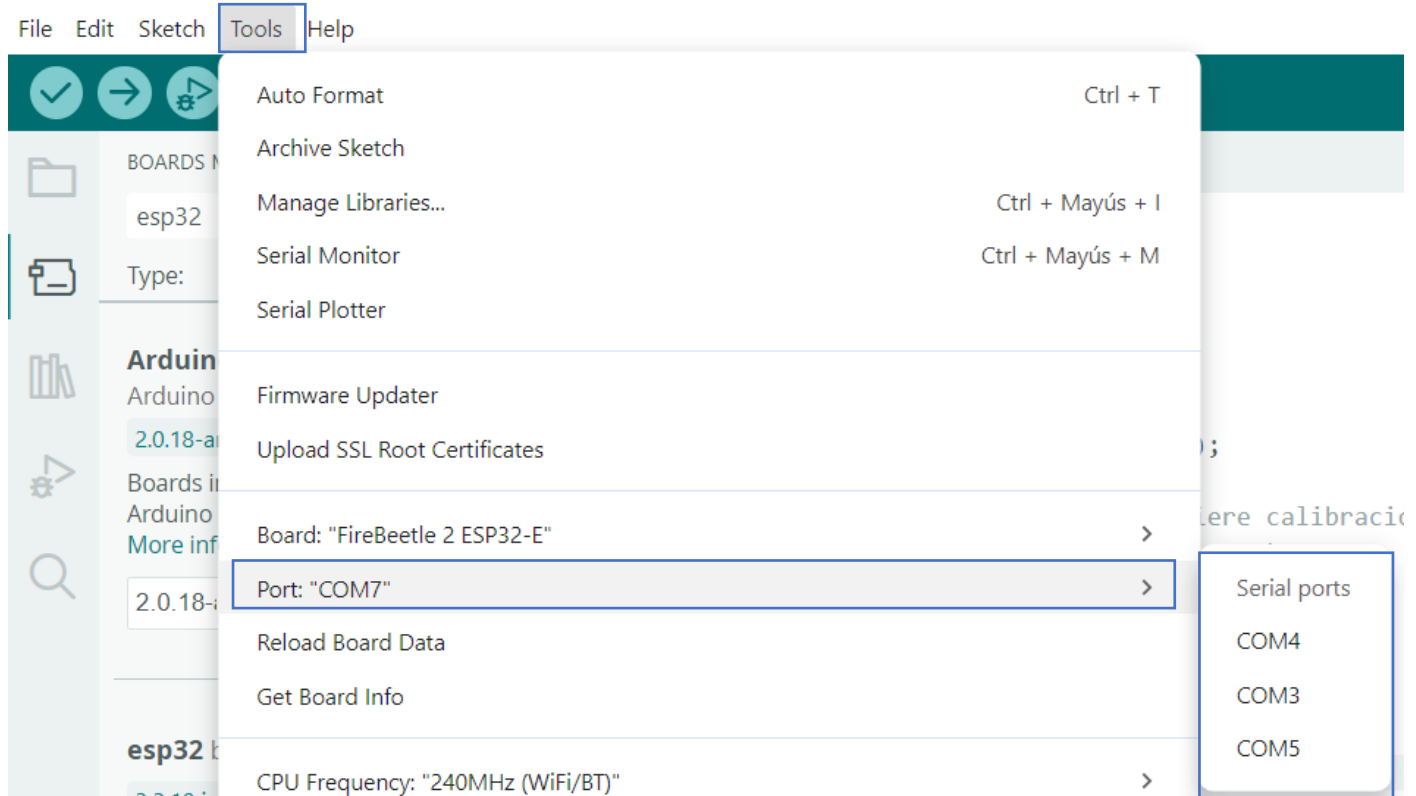


(Administrador de dispositivos nos vamos a Configuración de Windows y escribimos Administrador de dispositivos)

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

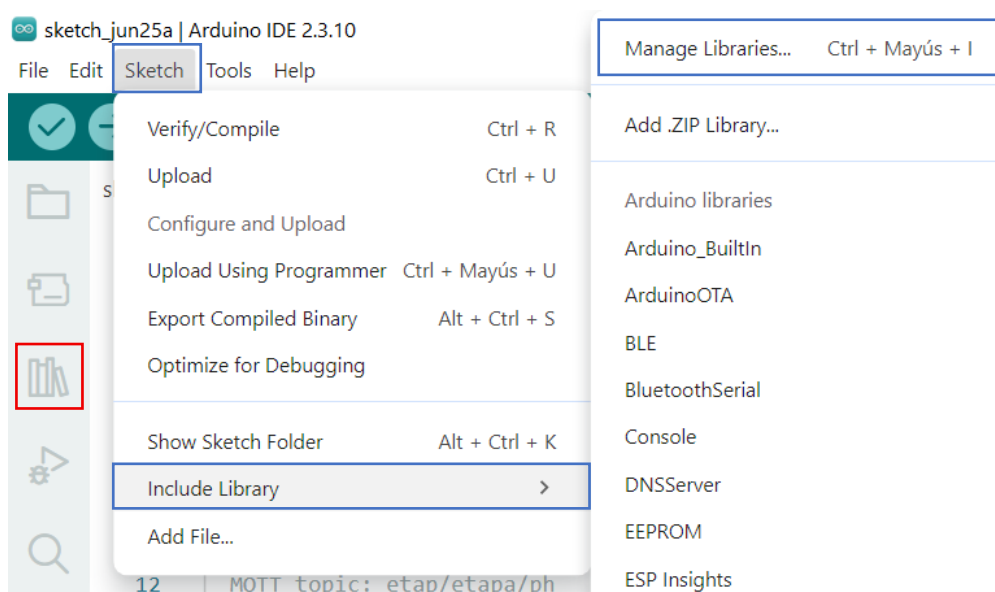
“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

-Seleccionar la conexión.



Debemos ir a Port y seleccionar el puerto en el que está conectada nuestra placa, los puertos que aparecen pueden variar en cada placa.

-Instalar la librería pubsubclient.



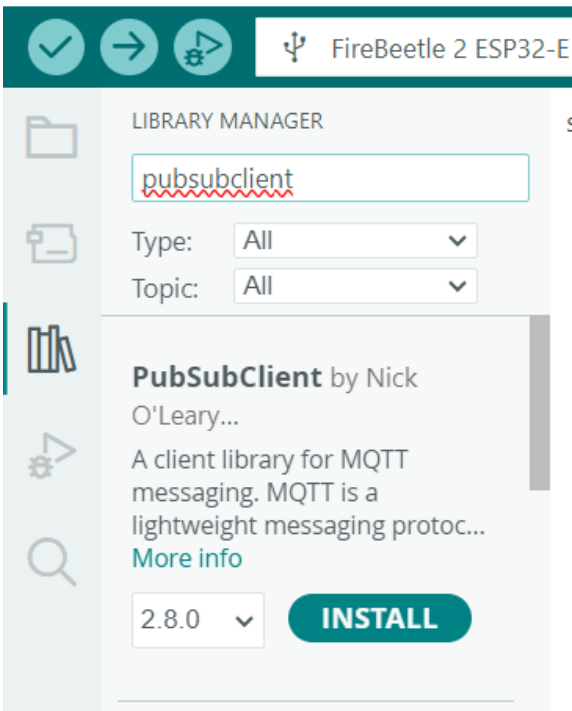
Nos vamos a Sketch, Incluye Library y pulsamos Manage Libraries... o pulsamos en el icono dentro del cuadro rojo.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

sketch_jun20a | Arduino IDE 2.3.10

File Edit Sketch Tools Help



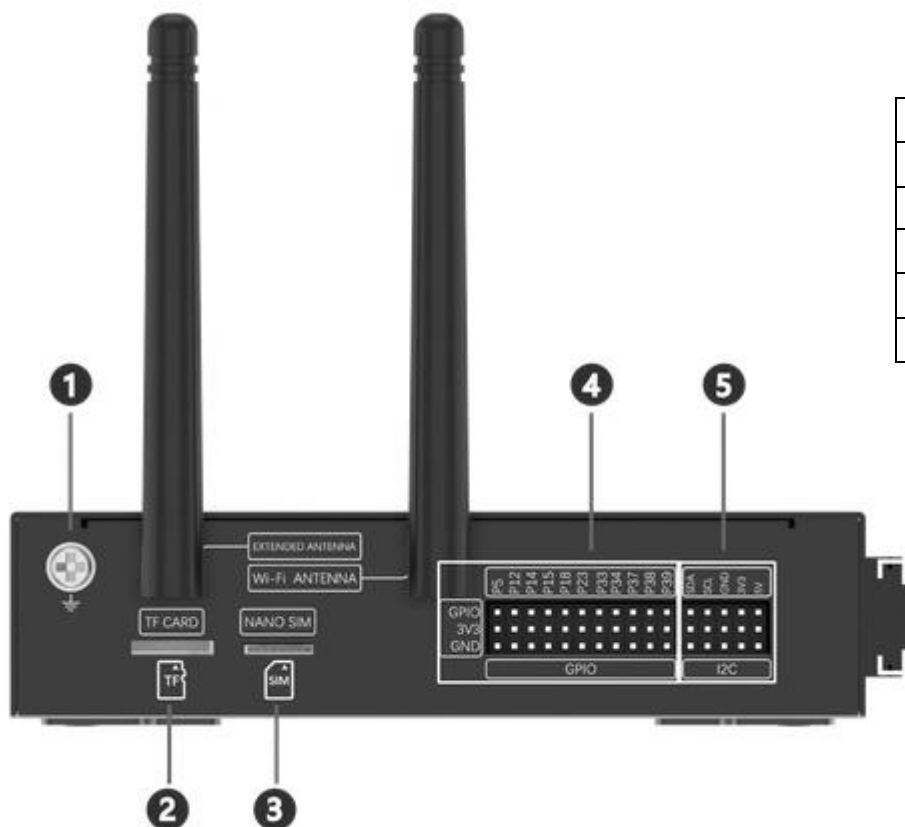
Escribimos pubsubclient y pulsamos INSTALL en la librería de Nick O'Leary.

Esta librería nos permitirá usar mqtt para la comunicación de la placa Firebeetle con la Raspberry PI.

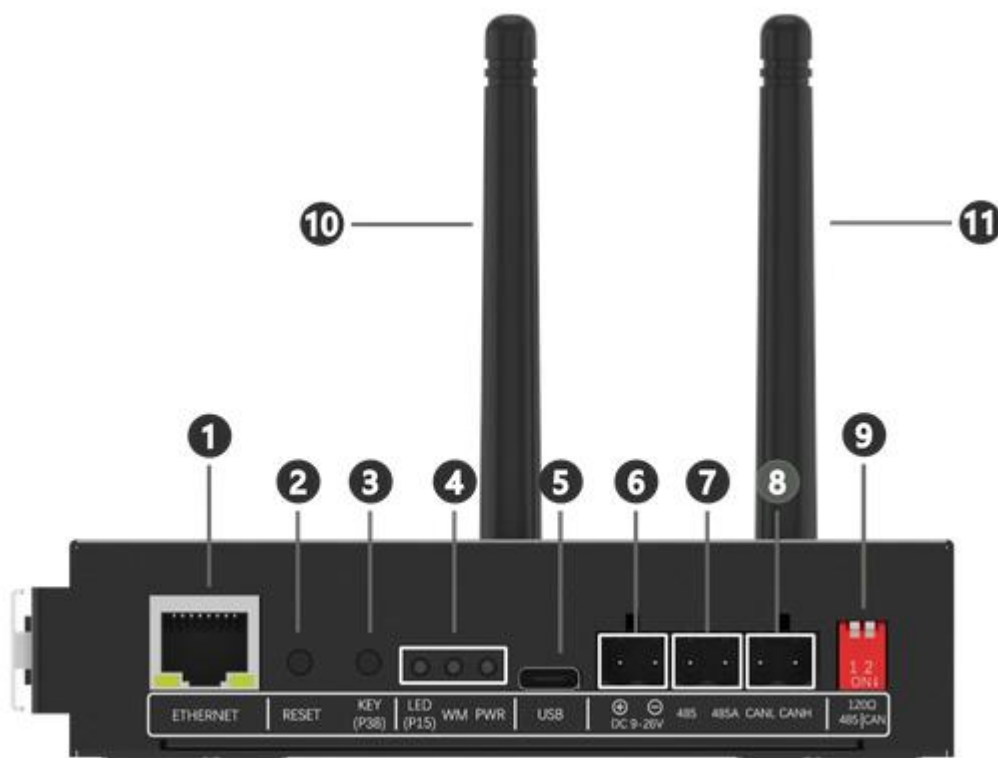
Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

EDGE 101



Ref	Descripción
1	Tierra
2	SD Card
3	Nano SIM
4	GPIO
5	I2C



Ref	Descripción
1	Ethernet
2	Reset
3	Botón definible
4	Indicadores led
5	USB-c
6	Alimentación 9-26v
7	RS485
8	CAN-BUS
9	Terminal
10	Antena wifi
11	Antena 4G

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

Información:

El **DFRobot DFR0886 Edge101** es un controlador IoT industrial basado en un **ESP32 de doble núcleo a 240 MHz**, diseñado para aplicaciones de automatización, adquisición de datos, monitorización remota y comunicaciones industriales. A diferencia de una placa ESP32 convencional, integra de serie interfaces industriales como **RS485, CAN Bus y Ethernet**, además de WiFi y Bluetooth, todo ello dentro de una carcasa metálica preparada para funcionamiento continuo 24/7.

Alimentación:

Podemos alimentarla con el puerto USB, o con una fuente de alimentación de tipo industrial 9-25v

Entradas:

-Digitales: 11 Entradas GPIO. GPIO05, GPIO12, GPIO14, GPIO15, GPIO18, GPIO23, GPIO33, GPIO34, GPIO37, GPIO38, GPIO39.

Tensión de funcionamiento: 0-3,3v.

-Analógicas: 11 Canales (ADC). ADC2_CH5(P12), ADC2_CH6(P14), ADC2_CH3(P15), ADC1_CH1(P37), ADC1_CH2(P38), ADC1_CH3(P39).

Para sensores analógicos usar preferentemente: P37, P38 o P39 porque pertenecen al ADC1 y continúan funcionando cuando el WiFi está activo

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

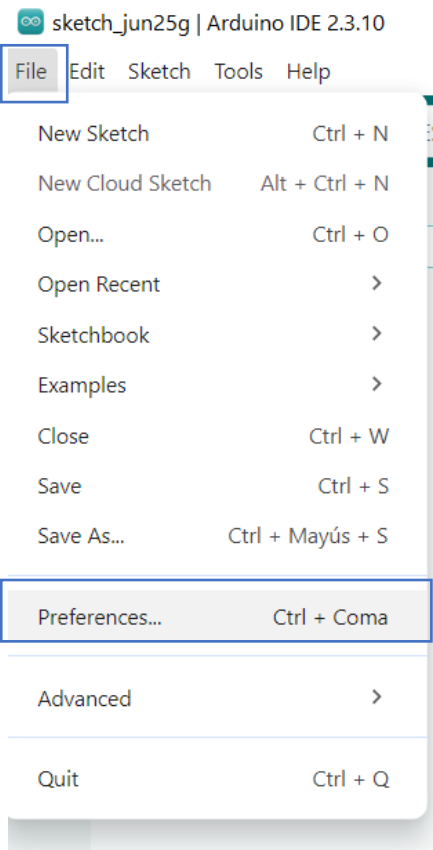
Pasos:

-Instalar el Arduino ide para programar la placa: <https://www.arduino.cc/en/Main/Software>

Downloads



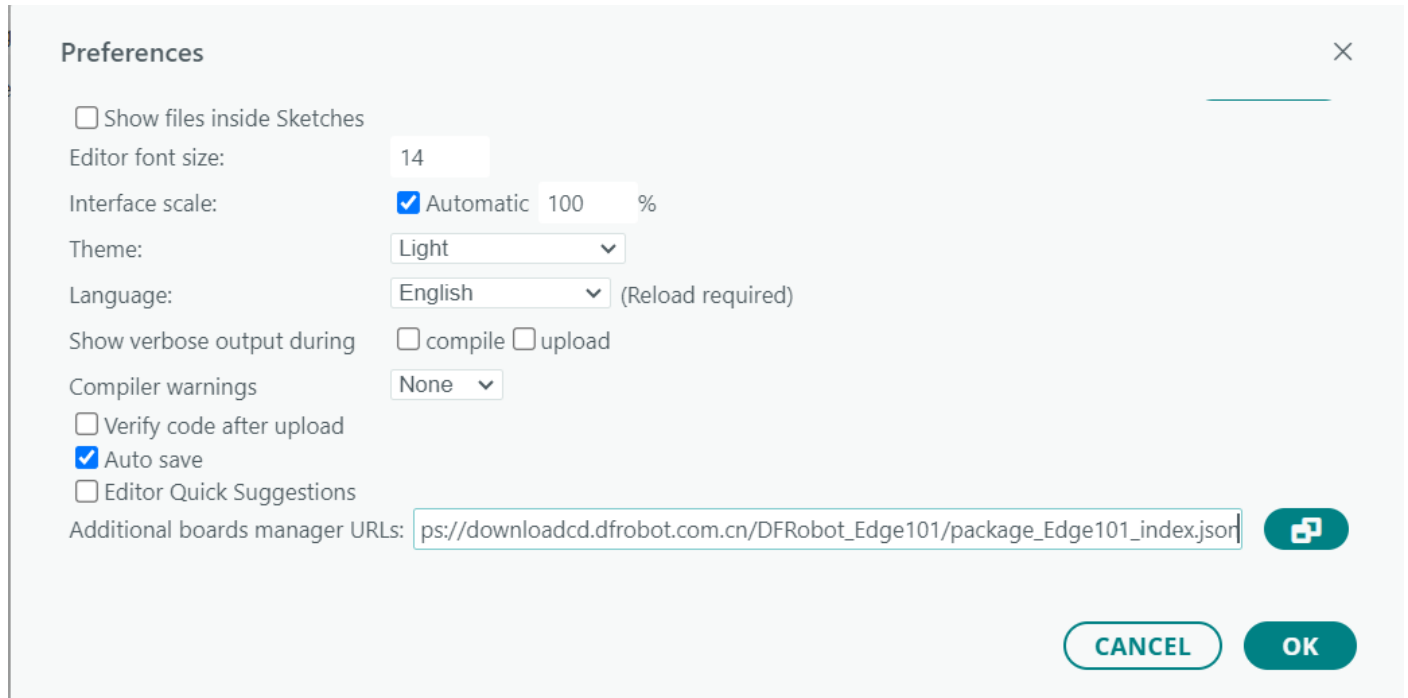
-Introducir en el Arduino ide los drivers de la placa Edge101:



Nos vamos a File, Preferences.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

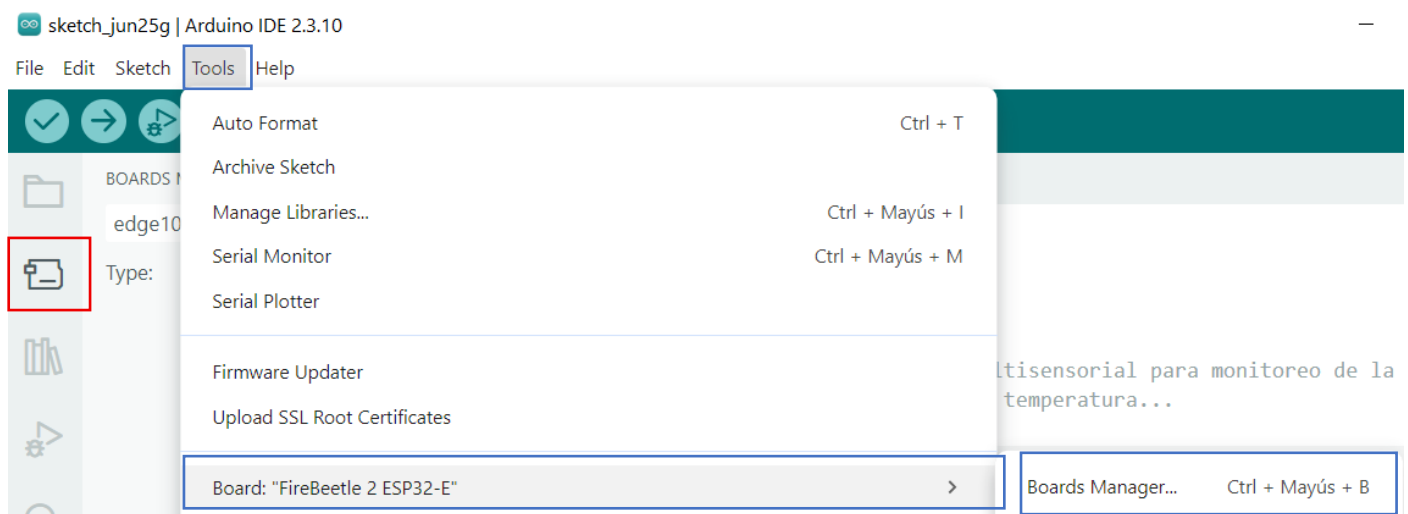


Pulsamos donde indica y seleccionamos le archivo de nuestra placa, para ello antes debemos descargarlo de:

https://downloadcd.dfrobot.com.cn/DFRobot_Edge101/package_Edge101_index.json

Pulsamos Ok.

-Instalar en Arduino IDE los drivers de nuestra placa:



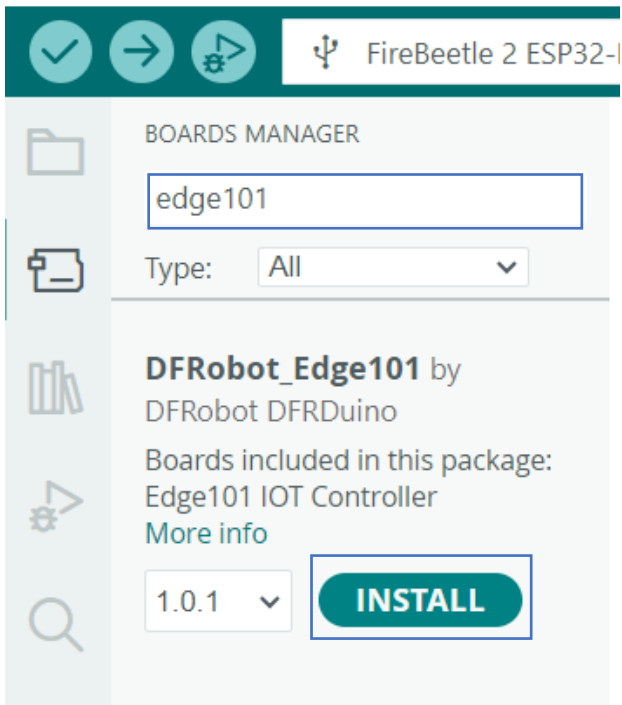
Nos vamos a Tools, Board: “Arduino XXX”, y seleccionamos Boards Manager, o pulsamos en el icono dentro del cuadro rojo.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

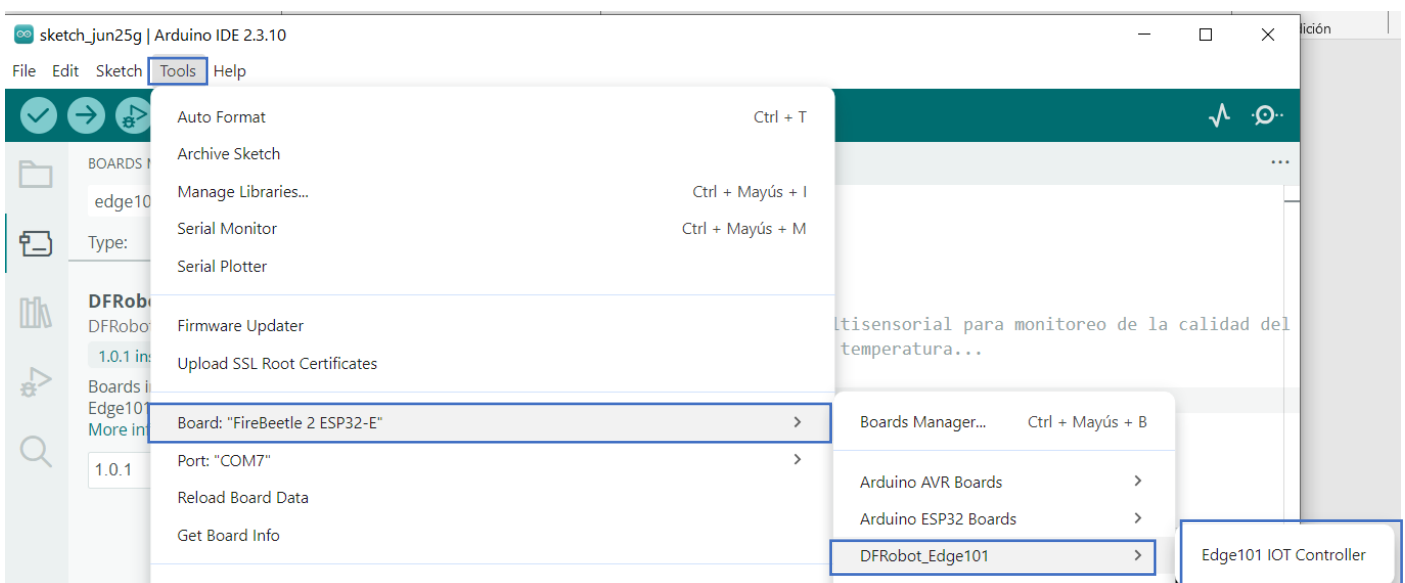
sketch_jun25g | Arduino IDE 2.3.10

File Edit Sketch Tools Help



Escribimos en el buscador EDGE101 y seleccionamos instalar en la que aparece en la imagen, este proceso puede tardar un poco.

-Seleccionar la placa.

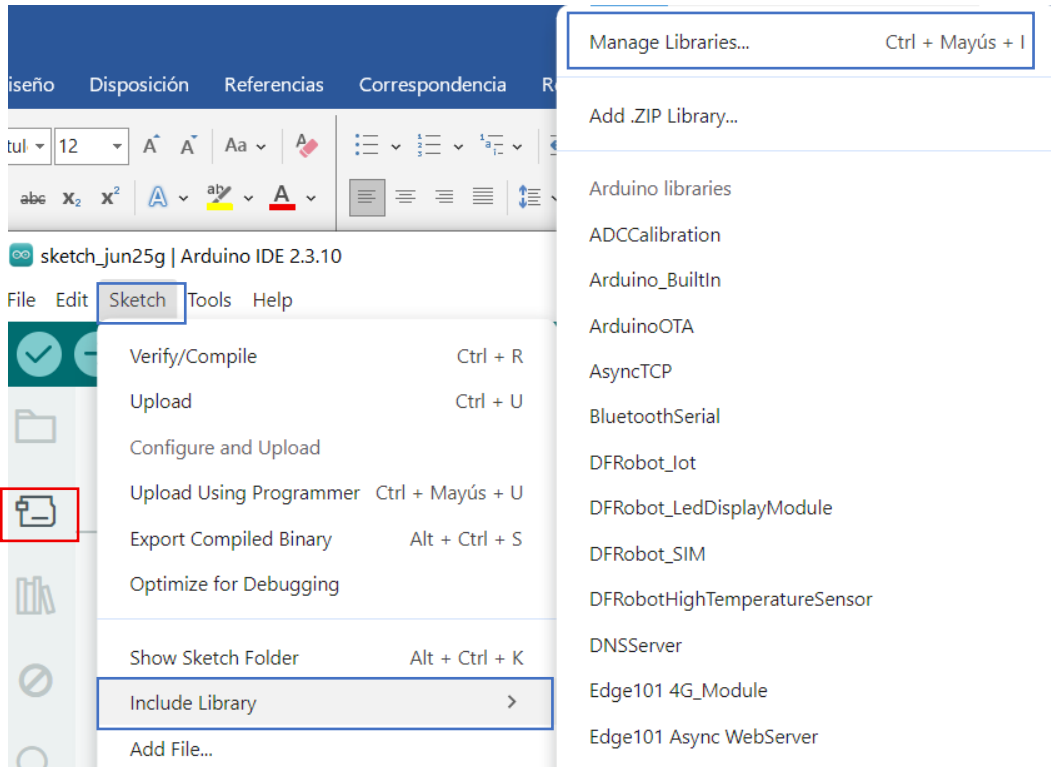


Nos vamos a Tools, Board, DFRobotEdge101 y seleccionamos Edge101 IOT Controller.

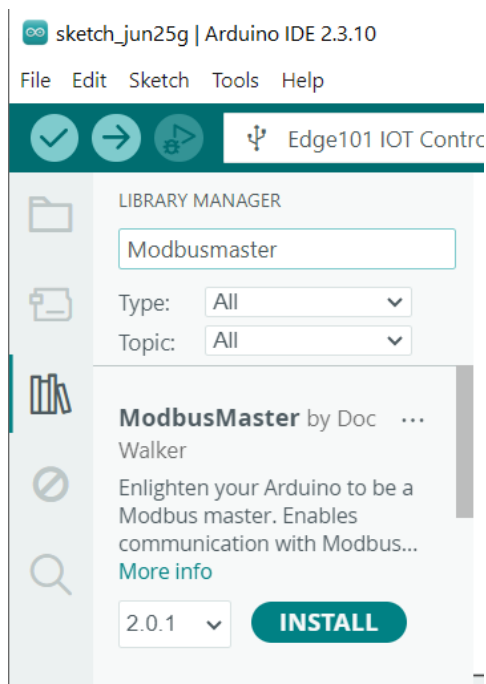
Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

-Instalar la librería ModbusMaster para la comunicación modbus.



Nos vamos a Sketch, Incluye Library, Manage Libraries o pulsamos el icono dentro del cuadro rojo.



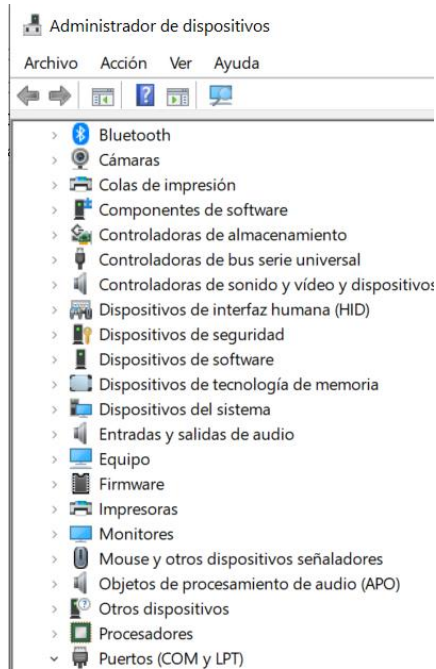
Escribimos Modbusmaster e instalamos la librería de Doc Walker.

-Conectar la placa Edge101 IOT Controller al ordenador.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

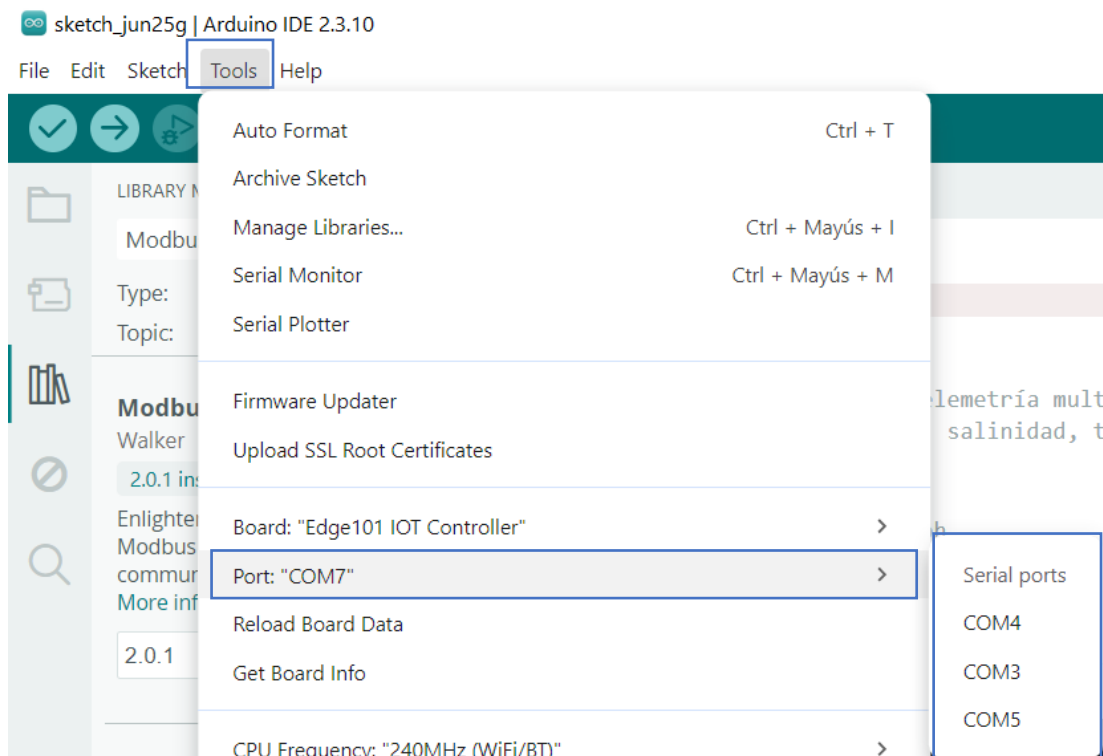
“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

-Debemos saber en que puerto COMM está conectada la Edge101 para ello nos vamos al administrador de dispositivos, buscamos Puertos (COM y LPT) y identificamos el que se corresponde con nuestra conexión.



(Administrador de dispositivos nos vamos a Configuración de Windows y escribimos Administrador de dispositivos)

-Seleccionar la conexión.

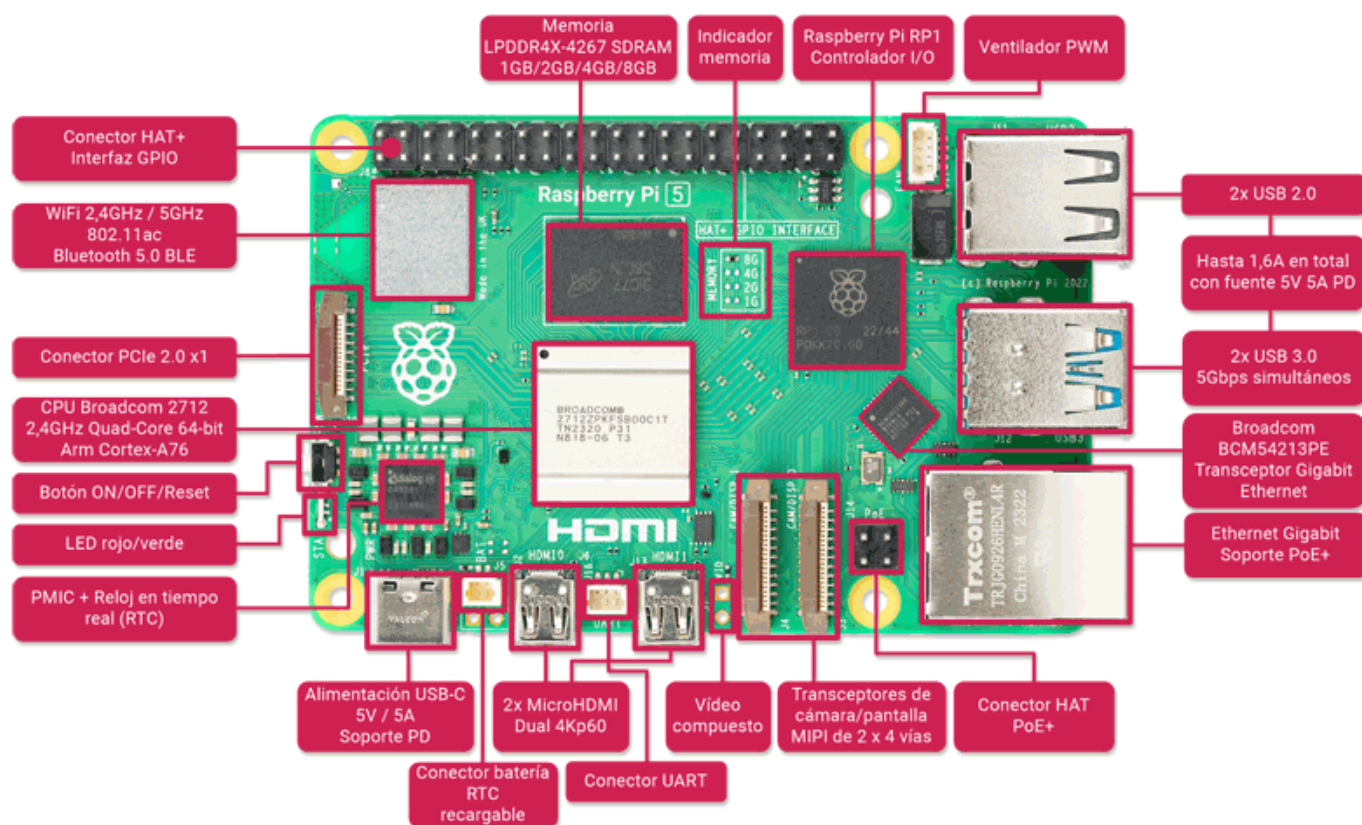


Debemos ir a Port y seleccionar el puerto en el que está conectada nuestra placa, los puertos que aparecen pueden variar en cada placa.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

RaspberryPI 5



Información:

La **Raspberry Pi** es un ordenador de placa reducida (*Single Board Computer, SBC*) desarrollado para aplicaciones educativas, de automatización, IoT y sistemas embebidos. A pesar de su pequeño tamaño, dispone de suficiente capacidad de procesamiento para ejecutar sistemas operativos completos basados en Linux, servidores web, bases de datos y plataformas de supervisión industrial.

En este proyecto, la Raspberry Pi actúa como **unidad central de adquisición y visualización de datos**, recibiendo la información enviada por los distintos FireBeetle ESP32 y el controlador industrial DFR0886. Además, almacena los datos históricos y proporciona una interfaz gráfica que permite supervisar en tiempo real el estado de la ETAP.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

Protocolo MQTT

MQTT (Message Queuing Telemetry Transport) es un protocolo de comunicación ligero diseñado para aplicaciones IoT y sistemas de adquisición de datos. Su principal ventaja es que consume muy poco ancho de banda y recursos de procesamiento, lo que lo hace ideal para dispositivos embebidos como los ESP32.

MQTT sigue una arquitectura basada en tres elementos:

Publicador (Publisher): dispositivo que envía información.

Broker: servidor que recibe y distribuye los mensajes.

Suscriptor (Subscriber): dispositivo o aplicación que recibe la información.

Funcionamiento

Sensor > ESP32 > MQTT Broker (Raspberry Pi) > Dashboard

Cada FireBeetle publica periódicamente las medidas de su sensor en un canal denominado **topic**. La Raspberry Pi recibe esos mensajes y los distribuye a las aplicaciones interesadas.

Por ejemplo:

- etap/ph
- etap/turbidez
- etap/tds
- etap/oxigeno
- etap/oxigeno_optico
- etap/saturacion
- etap/temperatura
- etap/salinidad
- etap/presion
- etap/nivel_deposito
- etap/nivel_tubo

Cuando un FireBeetle publica una medida en uno de estos topics, el broker MQTT la recibe y la envía automáticamente a los sistemas de visualización o almacenamiento.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

Pasos:

-Accedemos a la página de raspberry pi y descargamos el instalador de imágenes:

<https://www.raspberrypi.com/software/>

Raspberry Pi Imager

Raspberry Pi Imager is the quick and easy way to install **Raspberry Pi OS** and other operating systems to a microSD card, ready to use with your Raspberry Pi.

Download and install Raspberry Pi Imager on a computer with an SD card reader. Insert the microSD card you'll use with your Raspberry Pi into the reader and run Raspberry Pi Imager.

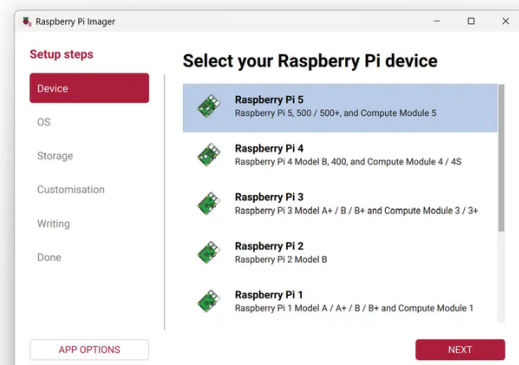
[Download for Windows](#)

[Read the documentation →](#)

[Download for macOS](#)

[Download for Linux \(x86_64\)](#)

To install on **Raspberry Pi OS**, type
`sudo apt install rpi-imager`
into a terminal window



-Insertamos la SD card en el ordenador.

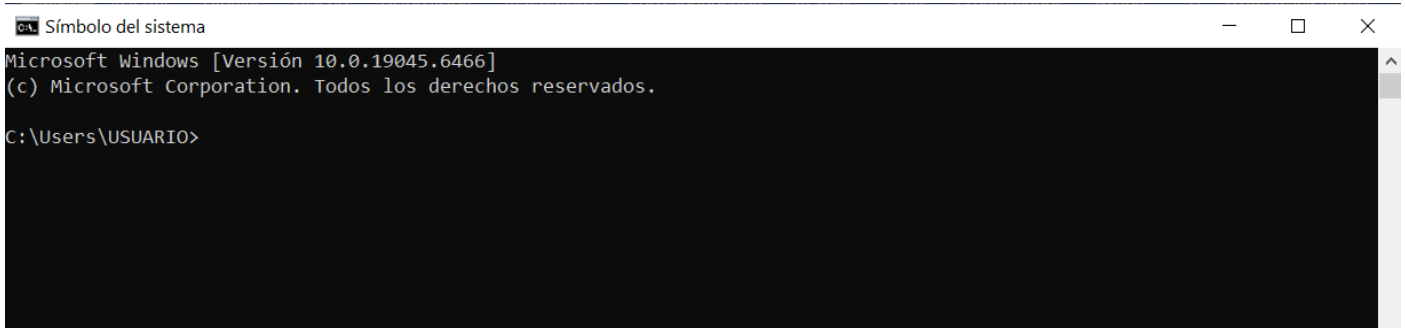
-Abrimos el raspberry pi imager y seleccionamos nuestra versión:



Seleccionamos Raspberry PI 5, Raspberry PI OS 64, la SD, en personalización seleccionamos nombre del equipo PI Etap contraseña “iesantoniogala”, localización España, configuración del wifi e introducimos la red de clase, acceso remoto activamos SSH, y pulsamos escribir.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua
“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

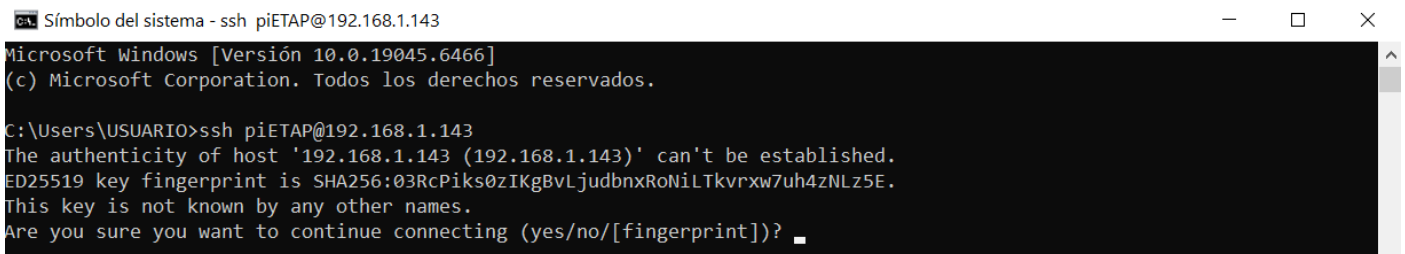
-Abrimos la consola de Windows estando conectados al wifi de la clase.



```
Símbolo del sistema
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USUARIO>
```

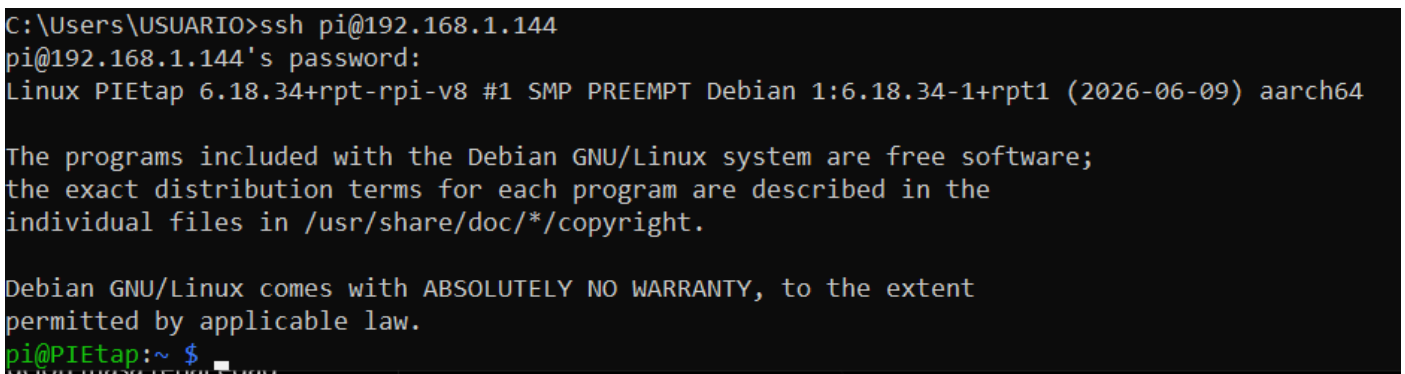
-Escribimos el siguiente comando: ssh piETAP@"ponemos la ip de la raspberry"



```
Símbolo del sistema - ssh piETAP@192.168.1.143
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USUARIO>ssh piETAP@192.168.1.143
The authenticity of host '192.168.1.143 (192.168.1.143)' can't be established.
ED25519 key fingerprint is SHA256:03RcPiks0zIKgBvLjudbnxRoNiLTkvrw7uh4zNLZ5E.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? _
```

Aceptamos las credenciales escribiendo yes y pulsando intro.



```
C:\Users\USUARIO>ssh pi@192.168.1.144
pi@192.168.1.144's password:
Linux PIETap 6.18.34+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.18.34-1+rpt1 (2026-06-09) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
pi@PIETap:~$
```

-Primero actualizaremos las raspberry:

Escribimos: sudo apt-update

Y cuando termine_ sudo apt-upgrade

Aceptamos los paquetes que nos pida escribiendo Y.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

-Instalación de bróker mqtt.

Escribimos: `sudo apt install mosquitto mosquitto-clients -y`

```
pi@PIEtap:~ $ sudo apt install mosquitto mosquitto-clients -y
Installing:
  mosquitto  mosquitto-clients

Installing dependencies:
  libldt2  libmosquitto1  libwebsockets19t64

Summary:
  Upgrading: 0, Installing: 5, Removing: 0, Not Upgrading: 0
  Download size: 741 kB
  Space needed: 2,156 kB / 52.6 GB available
```

Hacemos que arranque automáticamente:

```
sudo systemctl enable mosquitto
```

```
sudo systemctl start mosquitto
```

```
pi@PIEtap:~ $ sudo systemctl enable mosquitto
Synchronizing state of mosquitto.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable mosquitto
pi@PIEtap:~ $ sudo systemctl start mosquitto
```

-Instalación de node-red

Escribimos: `bash <(curl -sL https://raw.githubusercontent.com/node-red/linux-installers/master/deb/update-nodejs-and-nodered)`

Y aceptamos lo que nos pide pulsando y y enter.

Esperamos a que termine la instalación.

```
❏ Select a theme for the editor. To use any theme other than "default", you will need to install @node-red-contrib-theme-
s/theme-collection in your Node-RED user directory. · default

❏ Select the text editor component to use in the Node-RED Editor · monaco (default)

Node settings
=====
❏ Allow Function nodes to load external modules? (functionExternalModules) · Yes

Settings file written to /home/pi/.node-red/settings.js
pi@PIEtap:~ $
```

Configuramos para que node-red arranque automáticamente:

```
pi@PIEtap:~ $ sudo systemctl enable nodered.service
Created symlink '/etc/systemd/system/multi-user.target.wants/nodered.service' → '/usr/lib/systemd/system/nodered.service'.
pi@PIEtap:~ $ sudo systemctl start nodered.service
pi@PIEtap:~ $
```

Una vez terminado de instalar podremos acceder si estamos conectados a la misma red desde el navegador:

Escribimos: http://ip_raspberry:1880

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”



Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

Guía básica programación Arduino IDE.

Guía básica para comprender los códigos de los sensores

Objetivo

Esta guía explica los comandos y estructuras que aparecen en los programas utilizados durante las prácticas. No pretende enseñar todo Arduino, sino ayudar a comprender qué hace cada parte del código antes de cargarlo en la placa.

1. Librerías

Las librerías añaden funciones ya programadas.

Se incluyen al principio del programa mediante:

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ModbusMaster.h>
```

En nuestros programas:

WiFi.h → conecta la placa a la red WiFi.

PubSubClient.h → permite enviar datos mediante MQTT.

ModbusMaster.h → permite comunicarse con sensores RS485.

2. Definir constantes

Muchos valores aparecen definidos al inicio del programa.

Ejemplo:

```
#define PH_PIN 34
```

Significa que el sensor de pH está conectado al pin 34.

Otros ejemplos:

```
#define TDS_PIN 34
```

```
#define LEVEL_PIN 2
```

```
#define SENSOR_ID 1
```

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

3. Variables

Las variables almacenan información.

Ejemplos:

```
int adc;
```

Guarda números enteros.

```
float voltage;
```

Guarda números con decimales.

```
bool liquidDetected;
```

Guarda únicamente dos estados:

```
true
```

```
false
```

También aparecen:

```
char payload[200];
```

Que almacena texto para enviarlo por MQTT.

4. Configuración del WiFi

La red se define mediante:

```
const char* ssid = "TU_WIFI";
```

```
const char* password = "TU_PASSWORD";
```

Después se inicia con:

```
WiFi.begin(ssid, password);
```

Para comprobar si la conexión ha sido correcta se utiliza:

```
WiFi.status()
```

Mientras no exista conexión se repite:

```
while (WiFi.status() != WL_CONNECTED)
```


Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

5. Comunicación MQTT

El servidor MQTT se configura mediante:

```
client.setServer(mqtt_server, mqtt_port);
```

Para conectarse:

```
client.connect(...)
```

Para mantener la comunicación:

```
client.loop();
```

Para enviar información:

```
client.publish(topic, payload);
```

6. Comunicación Serie

Se utiliza para mostrar información en el ordenador.

Iniciar:

```
Serial.begin(115200);
```

Mostrar texto:

```
Serial.print("Voltaje:");
```

Mostrar una línea completa:

```
Serial.println(voltage);
```

Esto permite comprobar que el sensor está funcionando correctamente.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

7. Lectura de sensores

Entradas analógicas

Los sensores de:

pH

TDS

Turbidez

Oxígeno

utilizan:

```
analogRead(pin);
```

Ejemplo:

```
int adc = analogRead(PH_PIN);
```

El resultado es un número entre:

0 y 4095 (en ESP32 con resolución de 12 bits).

Entradas digitales

Los sensores de nivel utilizan:

```
digitalRead(pin);
```

Ejemplo:

```
digitalRead(LEVEL_PIN)
```

Solo pueden devolver:

HIGH

LOW

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

8. Configuración de pines

Los sensores digitales necesitan configurar el pin.

Ejemplo:

```
pinMode(LEVEL_PIN, INPUT);
```

o

```
pinMode(LEVEL_PIN, INPUT_PULLUP);
```

9. Configuración del conversor analógico

En varios sensores aparece:

```
analogReadResolution(12);
```

Configura el ADC para trabajar con:

0–4095

También:

```
analogSetPinAttenuation(...)
```

Permite adaptar el rango de voltajes que puede medir el ESP32.

10. Operaciones matemáticas

Los programas convierten la lectura del ADC en voltaje.

Ejemplo:

```
float voltage = adc * 3.3 / 4095.0;
```

Posteriormente ese voltaje se transforma en:

pH

NTU

ppm

mg/L

mediante fórmulas matemáticas.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

11. Condiciones

Las decisiones se realizan con:

```
if (...)
```

Ejemplo:

```
if (ntu < 0)
```

También:

```
if (liquidDetected) y else
```

12. Bucles

En los programas aparecen dos tipos.

while

Repite instrucciones mientras la condición sea verdadera.

Ejemplo:

```
while (WiFi.status() != WL_CONNECTED)
```

for

Se utiliza para recorrer arrays.

Ejemplo:

```
for(int i=0;i<number;i++)
```

Aparece en la función que calcula el promedio del sensor de pH.

13. Arrays

Un array almacena varios datos.

Ejemplo:

```
int pHArray[40];
```

En este caso se guardan las últimas 40 mediciones para obtener un valor más estable.

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

14. Tiempo

En lugar de medir continuamente, muchos programas esperan un intervalo.

Ejemplo:

```
millis()
```

Devuelve el tiempo desde que la placa se encendió.

Se utiliza así:

```
if (millis()- mqttTime > MQTT_INTERVAL)
```

De esta forma el envío MQTT se realiza cada cierto tiempo.

15. Esperas

Para hacer pausas:

```
delay(500);
```

Espera:

500 ms

16. Creación de mensajes

Antes de enviar datos por MQTT se crea un texto.

Ejemplo:

```
snprintf(payload,  
...  
);
```

El resultado es algo parecido a:

```
{  
  "sensor":"SEN0244",  
  "valor":235,  
  "unidad":"ppm"  
}
```

Proyecto: Sistema de telemetría multisensorial para monitoreo de calidad del agua

“Proyectos de innovación en digitalización aplicada (Medida 3.e.14) y Proyectos destinados a la innovación y transferencia del conocimiento (Medida 3.e.15).”

17. Envío MQTT

Una vez creado el mensaje:

```
client.publish(mqtt_topic, payload);
```

El dato queda disponible para otros equipos de la red.

18. Funciones auxiliares

En algunos programas aparecen bloques adicionales como:

```
setup_wifi()
```

Conecta el WiFi.

```
reconnect_mqtt()
```

Reconecta el servidor MQTT cuando se pierde la comunicación.

```
averageArray()
```

Calcula el promedio de varias muestras del sensor de pH para reducir el ruido.

```
publicarMQTT()
```

Evita repetir el mismo código al enviar diferentes variables del sensor SEN0681.